

Υλοποίηση κωδικοποιητή GSM 06.10 Συστήματα Πολυμέσων

Χρήστος Μάριος Περδίκης 10075 cperdikis@ece.auth.gr
Γιώργος Βακασίρης 9661 vakasiris@ece.auth.gr

Χειμερινό Εξάμηνο 2024-2025

Περίληψη

Αναφορά εργασίας για το μάθημα Συστήματα Πολυμέσων του Τμήματος Ηλεκτρολόγων Μηχανικών και Μηχανικών Υπολογιστών του ΑΠΘ τη χρονιά 2024-2025. Υλοποιήθηκε κωδικοποίηση φωνής σύμφωνα με το πρότυπο GSM 06.10. Υλοποιήθηκαν και τα τρία επίπεδα της εργασίας. Το πρόγραμμα επίδειξης της υλοποίησης δέχεται στην είσοδο ένα αρχείο wav. Το αρχείο χωρίζεται σε frames των 160 samples και πάνω σε αυτά πραγματοποιούνται οι δύο διαδικασίες pre-processing. Έπειτα το κάθε frame κωδικοποιείται, αποκωδικοποιείται, και τέλος δημιουργείται ένα νέο wav αρχείο. Αυτή είναι η ανακατασκευασμένη φωνή που θα ακούει ο δέκτης.

1 Εισαγωγή

Αυτή είναι η αναφορά εργασίας του μαθήματος Συστήματα Πολυμέσων στην οποία υλοποιήθηκε το πρότυπο κωδικοποίησης φωνής ETSI GSM 06.10. Υλοποιήθηκαν και τα τρία επίπεδα. Το zip αρχείο που υποβάλαμε περιέχει αυτή την αναφορά και τρεις φακέλους “1o_ερίπεδο”, “2o_ερίπεδο” και “3o_ερίπεδο”. Κάθε φάκελος περιέχει τον κώδικα του αντίστοιχου επιπέδου της εργασίας δηλαδή τον κωδικοποιητή / αποκωδικοποιητή, όλα τα βοηθητικά αρχεία και το πρόγραμμα επίδειξης. Ουσιαστικά από το ένα επίπεδο στο άλλο υπάρχουν αλλαγές μόνο στα αρχεία του κωδικοποιητή και αποκωδικοποιητή, encoder.py και decoder.py αντίστοιχα, και στο πρόγραμμα επίδειξης main.py μόνο στις εισόδους και εξόδους των συναρτήσεων κλήσης του κωδικοποιητή και αποκωδικοποιητή.

Για την εκτέλεση του προγράμματος επίδειξης του 3ου παραδοτέου είναι απαραίτητη η εγκατάσταση του python module bitstring.

2 Προετοιμασία — main.py

Το πρόγραμμά επίδειξης τρέχει με την εντολή “python main.py”. Το αρχείο main.py περιέχει τη λογική δομή του προγράμματος, από εκεί καλούνται όλες οι συναρτήσεις που υλοποιούν την κωδικοποίηση (και αποκωδικοποίηση) φωνής. Με τη βοήθεια της βιβλιοθήκης scipy αποθηκεύουμε όλα τα samples του αρχείου φωνής στο numpy array `audio_data_o`. Μπορεί ο χρήστης του προγράμματος να αλλάξει το όνομα του αρχείου εισόδου σε αυτό το σημείο. Οι διαδικασίες της κωδικοποίησης και της αποκωδικοποίησης πραγματοποιούνται frame ανά frame μέχρι το τέλος του αρχείου φωνής. Ένα frame αποτελείται από 160 samples. Ο αριθμός των επαναλήψεων που θα εκτελεστούν είναι ίσος με τον αριθμό των frames που θα επεξεργαστούν και είναι ίσος με $\lfloor \text{len}(\text{audio_data_o})/160 \rfloor$. Κάθε επανάληψη φορτώνεται το επόμενο frame στο `s_new`, numpy array με σταθερό μέγεθος 160 στοιχείων. Η μεταβλητή `offset` λειτουργεί σαν δείκτης στο πρώτο στοιχείο του επόμενου frame. Πάνω στα στοιχεία του `s_new` γίνεται η διαδικασία του preprocessing. Πρώτα καλούμε την συνάρτηση `offset_compensation()` και μετά την `pre_emphasis()` για να λάβουμε το array `s`. Η υλοποίηση των δύο συναρτήσεων υπάρχει στο αρχείο preprocessing.py. Στα στοιχεία του array `s` η κωδικοποίηση και μετά την αποκωδικοποίηση λαμβάνουμε ένα ανακατασκευασμένο `s0`. Στο τέλος κάθε επανάληψης ενημερώνουμε το αποθηκευμένο short term residual του προηγούμενου frame με το short term residual του τωρινού frame (`prev_frame_st_residual = curr_frame_st_residual`). Όταν ολοκληρωθούν όλες οι επαναλήψεις, κατασκευάζεται ένα νέο αρχείο ήχου “reconstructed_audio.wav” στο οποίο είναι ακουστή η παραμόρφωση που εισήγαγε η διαδικασία της κωδικοποίησης. Ο χρήστης του προγράμματος μπορεί να αλλάξει το όνομα του αρχείου εξόδου σε αυτό το σημείο.

Αν περισσέψουν samples του αρχείου φωνής εισόδου (ο αριθμός samples του δεν διαιρείται ακριβώς με το 160), τότε αυτά τα samples δεν συμμετέχουν στη διαδικασία της κωδικοποίησης και αγνοούνται. Θεωρούμε ότι είναι αποδεκτό να κόψουμε στη χειρότερη περίπτωση $159 \text{ samples}/8k\text{Hz} = 19.875\text{ms}$ ήχου παρά να αυξήσουμε την περιπλοκότητα του κώδικά μας.

2.1 Pre-processing

Οι συναρτήσεις pre-processing υλοποιούνται στο αρχείο `preprocessing.py` και δεν αλλάζουν στην μετέπειτα υλοποίηση των επιπέδων. Αφού έχει γίνει ο διαχωρισμός των frames, το κάθε frame εισάγεται διαδοχικά στις δύο συναρτήσεις του pre-processing, τις offset compensation και pre-emphasis. Η συνάρτηση που χρησιμοποιείται για το offset compensation σύμφωνα με το πρότυπο είναι η εξής:

$$S_{of}(k) = S0(k) - S0(k-1) + \alpha * S_{of}(k-1), \quad \alpha = 32735 * 2^{-15} \quad (1)$$

όπου $S0$ το αρχικό frame. Αντίστοιχα, η συνάρτηση που χρησιμοποιείται για το pre-emphasis:

$$s(k) = S_{of}(k) - \beta * S_{of}(k-1), \quad \beta = 28180 * 2^{-15} \quad (2)$$

Στην συνάρτηση `preprocessing.py`, οι παραπάνω σχέσεις χρησιμοποιούνται για να κατασκευαστούν φίλτρα με τους ανάλογους συντελεστές για την κάθε μια από τις παραπάνω διαδικασίες, τα οποία έπειτα εφαρμόζονται στο σήμα κάνοντας χρήση της συνάρτησης `lfilter` της βιβλιοθήκης `scipy.signal`. Προτιμήθηκε αυτός ο τρόπος υλοποίησης ώστε να μειωθεί η περιπλοκότητα του κώδικα που θα μπορούσαμε να έχουμε χρησιμοποιώντας `for-loops`. Για να κατασκευαστούν αυτά τα IIR φίλτρα, πρέπει να ορίσουμε κατάλληλα ποίοι θα είναι οι feedback και feedforward συντελεστές α και β ανάλογα με την περίπτωση, και να τους τοποθετήσουμε στα κατάλληλα σημεία της συνάρτησης `lfilter`. Λεπτομέρειες σχετικά με αυτό φαίνονται στην συνάρτηση `preprocessing.py`.

Η αντίστροφη διαδικασία του pre-processing, στην οποία αναφερόμαστε ως post-processing, υλοποιείται με αντίστοιχο τρόπο στα τελευταία στάδια του αποκωδικοποιητή `decoder.py`. Αυτό επίσης δεν αλλάζει στην μετέπειτα υλοποίηση των επόμενων επιπέδων. Οι αντίστροφες διαδικασίες βασίζονται στις σχέσεις σύμφωνα με το πρότυπο:

$$S_{of}[k] = S[k] + \beta * S_{of}[k-1] \quad (3)$$

$$S0[k] = S_{of}[k] + \alpha * S0[k-1] \quad (4)$$

Σημαντική είναι η σειρά με την οποία θα αντιστραφούν αυτές οι διαδικασίες, δηλαδή θα πρέπει το pre-emphasis να αντιστραφεί πρώτο εφόσον εφαρμόστηκε τελευταίο, και αντίστοιχα το offset compensation να αντιστραφεί τελευταίο. Στον κώδικα η υλοποίηση αυτών των διαδικασιών γίνεται πάλι με τη χρήση της συνάρτησης `lfilter`, μόνο που τα φίλτρα αυτή τη φορά είναι τα αντίστροφα από τα προηγούμενα.

3 Short Term Analysis — 1o επίπεδο

3.1 Κωδικοποιητής

Η υλοποίηση του κωδικοποιητή βρίσκεται στο αρχείο `encoder.py`. Η συνάρτηση αυτοσυσχέτισης υπολογίζεται σύμφωνα με την παράγραφο 3.1.4 του προτύπου ως:

$$r_s = \sum_{i=k}^{160} s[i] * s[i-k] \quad (5)$$

όπου $\mathbf{s}$ ένα array που περιέχει τα 160 samples του frame εισόδου και $\mathbf{rs}$ array με τις 9 τιμές της αυτοσυσχέτισης. Σχηματίζουμε τους πίνακες:

$$r = \begin{bmatrix} r_s[1] \\ r_s[2] \\ r_s[3] \\ r_s[4] \\ r_s[5] \\ r_s[6] \\ r_s[7] \\ r_s[8] \end{bmatrix}, R = \begin{bmatrix} r_s[0] & r_s[1] & r_s[2] & r_s[3] & r_s[4] & r_s[5] & r_s[6] & r_s[7] \\ r_s[1] & r_s[0] & r_s[1] & r_s[2] & r_s[3] & r_s[4] & r_s[5] & r_s[6] \\ r_s[2] & r_s[1] & r_s[0] & r_s[1] & r_s[2] & r_s[3] & r_s[4] & r_s[5] \\ r_s[3] & r_s[2] & r_s[1] & r_s[0] & r_s[1] & r_s[2] & r_s[3] & r_s[4] \\ r_s[4] & r_s[3] & r_s[2] & r_s[1] & r_s[0] & r_s[1] & r_s[2] & r_s[3] \\ r_s[5] & r_s[4] & r_s[3] & r_s[2] & r_s[1] & r_s[0] & r_s[1] & r_s[2] \\ r_s[6] & r_s[5] & r_s[4] & r_s[3] & r_s[2] & r_s[1] & r_s[0] & r_s[1] \\ r_s[7] & r_s[6] & r_s[5] & r_s[4] & r_s[3] & r_s[2] & r_s[1] & r_s[0] \end{bmatrix} \quad (6)$$

Λύνουμε τις κανονικές εξισώσεις $Rw = r$ ως προς w με τη συνάρτηση `numpy.linalg.solve`. Ο πίνακας w περιέχει τους 8 βέλτιστους συντελεστές $[a_1, a_2, \dots]$ του short term προβλέπτη. Πρέπει όμως να χρησιμοποιήσουμε τους συντελεστές που θα γνωρίζει ο αποκωδικοποιητής. Σχηματίζουμε ως είσοδο της βοηθητικής συνάρτησης `hw_utils.polynomial_coeff_to_reflection_coeff` το array **a_mod**:

$$a_mod = [1, -w[0], -w[1], -w[2], -w[3], -w[4], -w[5], -w[6], -w[7]] \quad (7)$$

Με τους συντελεστές ανάκλασης k_r υπολογίζουμε τα Log-Area Ratios (παράγραφος 3.1.6 εξίσωση 3.4 του προτύπου), τα κβαντίζουμε (παράγραφος 3.1.7) και κατόπιν τα αποκβαντίζουμε (παράγραφος 3.1.8). Για τον κβαντισμό και τον αποκβαντισμό των Log-Area Ratios υλοποιήθηκαν και οι βοηθητικές συναρτήσεις **Nint**, **A** και **B**, ο ορισμός τους βρίσκεται στο τέλος του αρχείου `encoder.py`. Μετατρέπουμε τα αποκωδικοποιημένα Log-Area Ratios σε αποκωδικοποιημένους συντελεστές ανάκλασης. Καλούμε την συνάρτηση `hw_utils.reflection_coeff_to_polynomial_coeff` με είσοδο το array **krd** των αποκωδικοποιημένων συντελεστών ανάκλασης, για έξοδο παίρνουμε τους αποκωδικοποιημένους συντελεστές του προβλέπτη **akd** σε μορφή παρόμοια με αυτή στην [εξίσωση 7](#):

$$akd = [1, -a_d[0], -a_d[1], -a_d[2], -a_d[3], -a_d[4], -a_d[5], -a_d[6], -a_d[7]] \quad (8)$$

και αντιστρέφουμε τα πρόσημα των στοιχείων **akd**[1...8], έτσι ώστε να περιέχουν τις τιμές των αποκβαντισμένων $a_1, a_2, \dots$. Τέλος λαμβάνουμε το residual **curr_frame_st_residual** εκτελώντας τη συνέλιξη μεταξύ των αρχικών samples **s** και της εξόδου του array **akd**, δηλαδή $d = s * akd$. Αυτό προκύπτει από το βήμα (β) 1 της παραγράφου 2.1.2 της εκφώνησης της εργασίας αλλά και από την παρακάτω απόδειξη:

$$\begin{aligned} \hat{s}(n) &= \sum_{k=1}^8 a_k s(n-k) \\ &= a_1 s(n-1) + a_2 s(n-2) + \dots \\ d(n) &= s(n) - \hat{s}(n) \\ &= s(n) - a_1 s(n-1) - a_2 s(n-2) - \dots \\ &= \sum_{k=0}^8 a'_k s(n-k) \end{aligned}$$

όπου $a'_k = akd$, το οποίο ισχύει για την περίπτωση που δεν αντιστρέψαμε τα πρόσημα των στοιχείων **akd**[1...8]. Παρόλα αυτά διαπιστώσαμε πειραματικά ότι αν τα αντιστρέψουμε το ηχητικό αποτέλεσμα είναι καλύτερο.

Δεν υλοποιήθηκε interpolation των τιμών των Log-Area Ratios στον κωδικοποιητή μας. Ο κωδικοποιητής του 1ου επιπέδου επιστρέφει τις ακόλουθες εξόδους στη σειρά:

- τα κωδικοποιημένα **LARc**,
- το **curr_frame_st_residual**

3.2 Αποκωδικοποιητής

Η υλοποίηση του αποκωδικοποιητή βρίσκεται στο αρχείο `decoder.py`, στην κύρια συνάρτηση `RPE_frame_st_decoder`. Σε αυτό το επίπεδο η συνάρτηση του αποκωδικοποιητή λαμβάνει ως εισόδους τα κωδικοποιημένα **LARc** και το

curr_frame_st_residual, όπως υπολογίστηκε από τον encoder του short term analysis. Αρχικά αποκωδικοποιούνται οι τιμές **LARc** σύμφωνα με την σχέση (3.7) της παραγράφου 3.1.8 του προτύπου. Στη συνέχεια, έχοντας τα αποκωδικοποιημένα **LAR**, τις χρησιμοποιούμε για να υπολογίσουμε τους συντελεστές **r** σύμφωνα με την σχέση (3.5) της παραγράφου 3.1.6 του προτύπου. Μέσω της συνάρτησης `reflection_coeff_to_polynomial_coeff` που μας δώθηκε, υπολογίζουμε τους συντελεστές **ak**. Στη συνέχεια χρησιμοποιώντας τους παραπάνω συντελεστές, κατασκευάστηκε το φίλτρο **H** ως εξής (σχέση 15 της εκφώνησης):

$$H_s(z) = \frac{1}{1 - \sum_{k=1}^8 \alpha_k z^{-k}} \quad (9)$$

Το φίλτρο αυτό λαμβάνει ως διέγερση την ακολουθία **curr_frame_st_resd** για να παράγει την αποκωδικοποιημένη εκδοχή του $s(n)$, το $s'(n)$. Στον κώδικα αναφερόμαστε σε αυτό ως **S**. Συγκεκριμένα στην υλοποίηση του φίλτρου στο `decoder.py`, οι συντελεστές του H_s υπολογίζονται με βάση την σχέση παραπάνω, εκτός από τον πρώτο συντελεστή $H_s[0]$, που ορίζεται ξεχωριστά ως 1. Αυτό γίνεται διότι όταν έχουμε $z = 0$, ο όρος του παρανομαστή γίνεται 1, και ταυτόχρονα διότι πρέπει να αποφύγουμε να υψώσουμε το 0 σε δύναμη. Παρόλα αυτά υπολογίζοντας τους συντελεστές του φίλτρου H_s η λειτουργία του δεν αλλάζει. Αφού συνελίξουμε το φίλτρο **H** με την ακολουθία **curr_frame_st_resd** χρησιμοποιώντας την συνάρτηση `convolve()` της `numpy`, έχουμε το σήμα **S**, που στη συνέχεια περνάει από τη διαδικασία `post-processing` όπως περιγράφηκε στην [υποενότητα 2.1](#). Τέλος ο αποκωδικοποιητής δίνει σαν έξοδο τη σήμα **S0**, δηλαδή το αποκωδικοποιημένο frame.

4 Long Term Analysis — 2ο επίπεδο

4.1 Κωδικοποιητής

Ο κώδικας που περιγράφουμε σε αυτήν την ενότητα ακολουθεί τον κώδικα της υλοποίησης του short term analysis στον κωδικοποιητή του 1ου επιπέδου. Η πρόβλεψη του pitch θα γίνει από το array **prev_d**. Είναι διαφορετικό για κάθε subframe και αποτελείται συνολικά από 120 στοιχεία. Στα πρώτα τρία subframes, επιτρέπουμε η πρόβλεψη να γίνει από το (ανακατασκευασμένο) short term residual του προηγούμενου frame. Πιο συγκεκριμένα και λαμβάνοντας υπόψιν ότι $N \in [40, 120]$, όταν βρισκόμαστε στο πρώτο subframe:

$$prev_d = prev_frame_st_residual[40 \dots 160] \quad (10)$$

δηλαδή το **prev_d** αποτελείται από τα τρία τελευταία subframes του προηγούμενου frame. Εφόσον η πρόβλεψη γίνεται από subframes παρελθοντικού frame, αυτά είναι ήδη ανακατασκευασμένα. Όταν βρισκόμαστε στο 2ο subframe:

$$prev_d = prev_frame_st_residual[80 \dots 160] + curr_frame_st_residual[0 \dots 40] \quad (11)$$

το **prev_d** αποτελείται από τα δύο τελευταία subframes του προηγούμενου frame και ακολουθεί το πρώτο — ανακατασκευασμένο πλέον — subframe του τωρινού frame. Με παρόμοιο τρόπο, στο 3ο subframe το **prev_d** αποτελείται από το τελευταίο subframe του προηγούμενου frame και τα δύο πρώτα subframes του τωρινού frame και στο 4ο subframe αποτελείται από τα 3 τρία προηγούμενα subframes του τωρινού frame. Προφανώς τα 40 τελευταία στοιχεία του **prev_d** δεν θα επιλεχθούν ποτέ για πρόβλεψη, εφόσον $N \geq 40$, υπάρχουν για να απλουστεύουν τη διαχείριση των indices.

Καλούμε τη συνάρτηση `RPE_subframe_slit_lte()` με εισόδους το τωρινό subframe **d** και το **prev_d** για να υπολογίσουμε τα **b**, **N**. Υπολογίζουμε τη συνέλιξη:

$$R(\lambda) = \sum_{i=0}^{39} d[i] * prev_d[120 + i - \lambda], \quad \lambda \in [40, 120] \quad (12)$$

Τα indices των **d**, **prev_d** προκύπτουν από την εξίσωση 3.9 του προτύπου. Για το **d**, $k_o = 0$ (όπου k_o το index του πρώτου στοιχείου του τωρινού frame) και αναφέρεται ήδη μόνο στο τωρινό subframe, άρα το $j * 40$ είναι περιττό. Για το **prev_d**, $k_j = 120$ πάντα, εφόσον το **prev_d** είναι ουσιαστικά ένα sliding window που για κάθε τωρινό subframe περιέχει τα τρία προηγούμενα subframes. Ελέγχουμε ποιό $R(\lambda)$ είναι η μέγιστη και θέτουμε ως **N** το λ που τη μεγιστοποίησε. Έπειτα υπολογίζουμε το **b** σύμφωνα με την εξίσωση 3.11 του προτύπου. Ο αριθμητής

του $\mathbf{b}$ είναι ίσος με $R(N)$ και ο παρανομαστής του $\sum_{i=0}^{39} (prev_d[120 + i - N])^2$, βρίσκουμε το $\mathbf{b}$ διαιρώντας τον αριθμητή με τον παρανομαστή.

Τα $\mathbf{N}, \mathbf{b}$ που επιστρέφει η `RPE_subframe_sl_tle()` αποθηκεύονται το καθένα σε μια λίστα τεσσάρων στοιχείων $N[j], b[j]$ ένα στοιχείο για κάθε διαφορετικό subframe. Κβαντίζουμε τα $\mathbf{N}[j], \mathbf{b}[j]$ (κάθε φορά μόνο το στοιχείο του τωρινού subframe). Το $\mathbf{N}[j]$ είναι ήδη ένας ακέραιος οπότε είναι ήδη κβαντισμένος. Το $\mathbf{b}[j]$ κβαντίζεται σύμφωνα με την εξίσωση 3.15 του προτύπου σε $\mathbf{bc}[j]$. Υλοποιήσαμε τις βοηθητικές συναρτήσεις `QLB()` και `DLB()`, όπως αυτές περιγράφονται από τον πίνακα 3.3 του προτύπου. Οι ορισμοί των `QLB()` και `DLB()` βρίσκονται στο τέλος του αρχείου `encoder.py`.

Για να υλοποιήσουμε το στάδιο της πρόβλεψης αρχικά αποκβαντίζουμε το $\mathbf{bc}[j]$ σε $\mathbf{bd}[j]$ με τη βοήθεια της `QLB`. Υπολογίζουμε την πρόβλεψη $\mathbf{d_predict}$:

$$d_predict[i] = \sum_{i=0}^{39} bd[j] * prev_d[120 + i - N[j]] \quad (13)$$

και το long term residual $\mathbf{e}$:

$$e[j * 40 + i] = d[i] - d_predict[i] \quad (14)$$

Το array $\mathbf{e}$ περιέχει το σφάλμα και για τέσσερα subframes του frame, για αυτό είναι αναγκαία η προσθήκη του offset $j * 40$, όπου j προσδιορίζει σε ποιο subframe βρισκόμαστε (για $j = 0$ βρισκόμαστε στο πρώτο subframe του τωρινού frame, για $j = 1$ στο δεύτερο κλπ.). Τέλος, υπολογίζουμε το ανακατασκευασμένο short term residual $\mathbf{d_reconstruct}$ από το $\mathbf{e}$:

$$d_reconstruct[j * 40 + i] = e[j * 40 + i] + d_predict[i] \quad (15)$$

Ο κωδικοποιητής του 2ου επιπέδου επιστρέφει με τη σειρά

- τα κωδικοποιημένα **LARc**,
- το **d_reconstruct** ως **curr_frame_st_residual**
- τη λίστα **N**
- τη λίστα κβαντισμένων **bc**
- το long term residual **e** ως **curr_frame_ex_full**

4.2 Αποκωδικοποιητής

Σε αντίθεση με τον κωδικοποιητή, η υλοποίηση του 2ου επιπέδου στον αποκωδικοποιητή βρίσκεται πριν την υλοποίηση του 1ου επιπέδου. Η είσοδος του αποκωδικοποιητή είναι όλες οι έξοδοι του κωδικοποιητή μαζί με το **prev_frame_st_residual**. Ξεκινάμε αποκβαντίζοντας το $\mathbf{b}$ με την βοηθητική συνάρτηση `QLB()` (βρίσκεται και στο τέλος του `decoder.py`). Το $\mathbf{N}$ είναι ήδη αποκβαντισμένο εφόσον είναι ακέραιος. Ξανακατασκευάζουμε το **prev_d** όπως περιγράψαμε στην [υποενότητα 4.1](#) αυτής της αναφοράς. Υπολογίζουμε το **d_predict** από την [εξίσωση 13](#), και έπειτα το **d_reconstruct** από την [εξίσωση 15](#), με τη μόνη διαφορά ότι χρησιμοποιούμε το **curr_frame_ex_full** αντί για το **e**. Όσον αφορά το 2ο επίπεδο, τα δύο array έχουν τα ίδια στοιχεία, απλά έχουν διαφορετική ονομασία. Τέλος, θέτουμε **curr_frame_st_residual = d_reconstruct**, έτσι ώστε ο αποκωδικοποιητής του 1ου επιπέδου να χρησιμοποιήσει το ανακατασκευασμένο short term residual.

5 Bitstream και RPE — 3ο επίπεδο